

Búsqueda de Ruta Mínima en Grafos con OpenMP

Examen Parcial 1 — Computación Paralela

Consultora HPC: Tres Punto Uno Cuatro

Anthony Lou Schwank • Christa Isabella Obando

11 de septiembre de 2026

<https://github.com/anthonylouschwank/Parcial1-Paralela>

1. Contexto y datos

El problema

Dada una red social de millones de usuarios (nodos) y sus amistades (aristas), hay que encontrar el camino más corto de conexiones entre un Usuario X y un Usuario Y, explorando vecinos paso a paso mediante una cola de tareas pendientes.

La dificultad no es el algoritmo, que es un BFS de libro. Es el **desbalance de carga**: en una red social los grados siguen una ley de potencias. Casi todos los usuarios tienen un puñado de amigos, pero unos pocos *hubs* tienen decenas de miles. Repartir la exploración ingenuamente deja a un hilo terminando en microsegundos mientras otro procesa solo las 10 000 amistades de un influencer.

Propuesta secuencial

Implementamos dos variantes en `/secuencial`:

1. **BFS de cola** (`bfsCola`): el algoritmo de libro con `std::queue`. Es la “cola de tareas pendientes” literal del enunciado y sirve como referencia de correctitud.
2. **BFS por frontera** (`bfsFrontera`): la misma exploración, pero la cola se reorganiza en *frontera actual* y *frontera siguiente*. Dentro de un nivel los usuarios son independientes entre sí, y ahí está el paralelismo.

La segunda es la *baseline* contra la que medimos el speedup, porque es el mismo algoritmo que corre la versión paralela. Comparar contra otra cosa inflaría el resultado artificialmente.

Justificación de los datos de prueba

Origen. Generamos la red con el modelo de conexión preferente de Barabási–Albert: cada usuario nuevo elige amigos con probabilidad proporcional a los que ya tienen. Preferimos esto a descargar un dataset real por dos razones prácticas: la semilla fija (314) garantiza que todas las máquinas midan sobre la red *idéntica*, y podemos escalar el tamaño a voluntad. El generador vive en `/comun`, compartido por ambas implementaciones, para que no haya forma de que midan sobre grafos distintos.

Tamaño. 2 000 000 de usuarios y 15 999 964 amistades. El CSR ocupa 137.3 MB.

¿Reproduce el fenómeno? Sí, y está medido, no supuesto:

	Mínimo	Mediana	Promedio	p99	Máximo
Amistades por usuario	8	11	16.0	84	6 414

El 1 % de usuarios más conectados concentra el 10.5 % de las amistades, y la razón máximo/mediana es de **583**×. Ese es exactamente el desbalance que hay que vencer. Con 5 000 000 de usuarios el hub máximo llega a **10 249** amigos, el número literal del enunciado.

Estructura en memoria. Grafo no dirigido en formato **CSR** (*Compressed Sparse Row*): un arreglo `offsets` de tamaño $n + 1$ y uno `ady` de tamaño $2m$ con toda la adyacencia contigua. Frente a un `vector<vector<int>>`, CSR evita dos millones de asignaciones dispersas en el heap y un fallo de caché por cada salto de nodo. Además es de solo lectura durante el BFS, así que todos los hilos la comparten sin ninguna sincronización.

2. Estrategia de paralelización

Directivas utilizadas

#pragma omp parallel, una sola vez para todo el BFS. Nuestra primera versión abría y cerraba una región paralela *por nivel*. Los niveles 0–2 mueven apenas unos miles de aristas, pero costaban 20 ms con 8 hilos contra 0.45 ms con 1: puro costo de arrancar y dormir el equipo de hilos. Ahora el equipo se crea una vez y los niveles se separan con barreras, que son órdenes de magnitud más baratas.

#pragma omp for schedule(runtime) sobre la frontera. Cada iteración expande un usuario, y su costo es proporcional a su número de amigos, que varía en tres órdenes de magnitud. Usamos `runtime` y no una política fija para que las tres alternativas recorran exactamente el mismo código máquina: así la comparación entre ellas mide la política y no diferencias de compilación.

#pragma omp single y #pragma omp barrier para cerrar cada nivel: calcular desplazamientos, intercambiar fronteras y decidir si continuar.

Cómo evitamos las condiciones de carrera

Dos hilos pueden descubrir al mismo usuario w en el mismo instante. Si ambos leen `visitado[w]` == 0 y ambos lo encolan, w entra duplicado en la frontera y `padre[w]` queda indefinido.

La solución es un **compare-and-swap atómico** sobre `visitado[w]`. Exactamente un hilo logra la transición $0 \rightarrow 1$; ese es el único que escribe `padre[w]` y encola. Es indivisible: no hay ventana entre “leo que está libre” y “lo marco como mío”. Lo precede una lectura relajada como filtro barato, porque la enorme mayoría de vecinos ya fueron visitados y no vale la pena pagar el CAS.

Un segundo punto de contención es la construcción de la frontera siguiente. Si todos los hilos hicieran `push_back` sobre un vector compartido — o peor, dentro de un `critical` — se serializarían millones de inserciones por nivel. En vez de eso **cada hilo acumula en su propio buffer local**, sin sincronización alguna, y al cerrar el nivel se calculan los desplazamientos y cada hilo copia su bloque en paralelo a un rango disjunto.

Verificación. El programa comprueba en cada corrida que el camino hallado sea real (cada salto debe ser una amistad existente) y que las invariantes — usuarios visitados y amistades recorridas — no varíen entre repeticiones. Si hubiera una carrera sin resolver, variarían. En las 52 corridas de las dos máquinas no varió ninguna.

Sí cambia el *camino* entre corridas, y es correcto: gana el hilo que consiga el CAS, y es él quien queda como padre. La *longitud* nunca cambia.

Cómo atacamos el desbalance de carga

Probamos las tres políticas de OpenMP con cuatro tamaños de bloque, en las dos máquinas, con 8 hilos:

Scheduling	Chunk	i5-1035G1 (ms)	i7-11370H (ms)
static	default	140.44	136.06
static	16	141.02	121.31
static	64	150.87	130.05
static	256	162.74	180.34
dynamic	default	152.76	140.17
dynamic	16	130.18	109.64
dynamic	64	139.03	112.04
dynamic	256	172.32	112.41
guided	default	143.61	119.12
guided	16	143.53	124.33
guided	64	148.30	117.52
guided	256	144.76	128.78

dynamic con chunk 16 gana en las dos máquinas de forma independiente, por 7.9 % sobre el mejor **static** en la i5 y 10.6 % en la i7. Un resultado replicado en hardware distinto es mucho más sólido que uno medido una sola vez. El chunk pequeño importa: con 256 el reparto se vuelve grueso y **dynamic** pierde contra **static**.

Un matiz honesto. La ganancia es real pero modesta, y conviene decir por qué. En los niveles dominantes la frontera tiene más de un millón de usuarios; repartida entre 8 hilos, la varianza de los grados se promedia sola. El desbalance muerde en los niveles intermedios: en el nivel 3 medimos $1.45\times$ de desbalance incluso con **dynamic**, y hasta $2.06\times$ en el nivel 2 con **static**.

Además, nuestra métrica de desbalance (aristas del hilo más cargado sobre el promedio) *engaña* con **dynamic**: llega a marcar $1.38\times$ contra $1.09\times$ de **static** y aun así **dynamic** gana de punta a punta. La razón es que **dynamic** no iguala aristas asignadas, iguala **tiempo**: un hilo que corre más lento toma menos bloques, que es exactamente lo que debe pasar.

3. Resultados y métricas

Metodología

Ambos binarios se compilan con banderas idénticas (`-O3 -std=c++17`); la única diferencia es `-fopenmp`. Se reporta la **mediana** de 7 repeticiones, no el promedio, porque en una laptop cualquier tarea del sistema operativo produce valores atípicos.

Entre configuraciones hay una **pausa de 30 segundos**. Medirlo nos costó una tarde: sin la pausa, la misma configuración daba 132 ms en frío y 316 ms tras diez minutos de carga sostenida. Sin enfriar, la curva de escalabilidad mide la disipación térmica del equipo, no el algoritmo.

Cada corrida deja su salida completa en `docs/resultados/<máquina>/salidas/`, y los tiempos crudos en `mediciones.csv` (196 filas por máquina).

Escenario principal: recorrido completo de la red

Las dos versiones recorren exactamente las mismas 31 999 928 amistades y visitan los 2 000 000 de usuarios, así que la comparación es limpia.

Christa Isabella Obando Guzman — Intel i5-1035G1

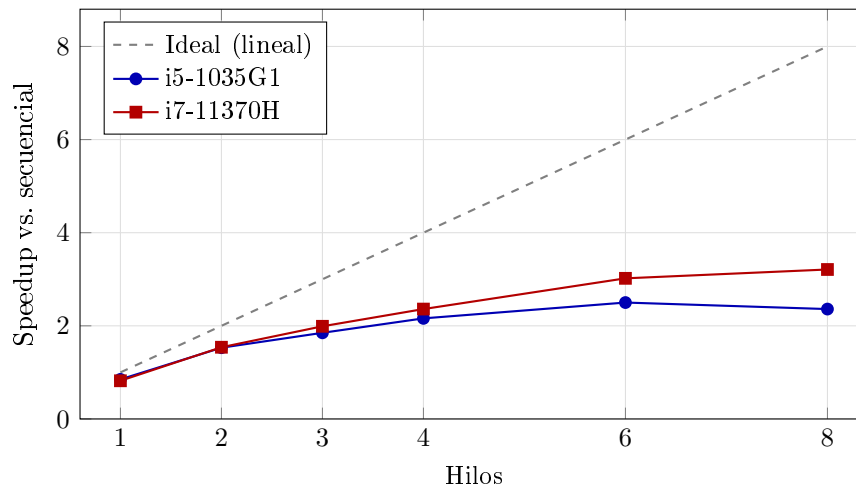
4 núcleos físicos / 8 hilos lógicos, 15 W. Baseline secuencial: **342.54 ms**.

Hilos	Mediana	Speedup	Efic.
1	400.91	0.85×	85 %
2	224.33	1.53×	76 %
3	184.66	1.85×	62 %
4	158.79	2.16×	54 %
6	137.04	2.50×	42 %
8	144.89	2.36×	30 %

Anthony Lou Schwank — Intel i7-11370H

4 núcleos físicos / 8 hilos lógicos, 35 W. Baseline secuencial: **369.19 ms**.

Hilos	Mediana	Speedup	Efic.
1	449.46	0.82×	82 %
2	240.40	1.54×	77 %
3	185.71	1.99×	66 %
4	156.34	2.36×	59 %
6	122.22	3.02×	50 %
8	114.86	3.21×	40 %



Lectura de los resultados

Con 1 hilo el speedup es menor a 1 ($0.85\times$ y $0.82\times$). Es el costo de arrastrar el runtime de OpenMP sin ningún beneficio, y explica de dónde parte la curva.

La eficiencia cae porque se acaban los núcleos, no porque falle el reparto. Ambas máquinas tienen 4 núcleos físicos; más allá de ese número los hilos comparten unidades de ejecución (SMT) y no hay más núcleos entre los que repartir. A 4 hilos la eficiencia es 54 % y 59 %.

Mismo paralelismo, techo distinto. Las dos máquinas tienen idéntico número de núcleos, pero la i5 hace pico en 6 hilos y *empeora* con 8, mientras la i7 sigue mejorando. Como el grado de paralelismo es el mismo, la diferencia no puede ser del algoritmo: es ancho de banda de memoria y frecuencia sostenida. Esto confirma que el BFS aquí está limitado por memoria y no por cómputo, algo que detectamos en la versión inicial al medir nivel por nivel: el nivel dominado por escrituras escalaba $1.19\times$ mientras el dominado por lecturas escalaba $1.87\times$.

Limitación medida: la consulta $X \rightarrow Y$

En la consulta puntual del enunciado nuestra versión paralela **pierde** contra el BFS de cola secuencial:

	BFS de cola (1 hilo)	Paralelo (8 hilos)	Speedup real
i5-1035G1	49.66 ms	59.97 ms	$0.83\times$
i7-11370H	37.73 ms	52.91 ms	$0.71\times$

La causa está medida. El BFS de cola abandona *en el instante* en que descubre el destino, a media capa, y recorre 1 599 662 aristas. Nuestra versión por frontera debe cerrar el nivel completo antes de poder revisar, y ese nivel es justo donde la frontera explota: recorre 12 450 125 aristas, **7.8× más trabajo**. Paralelizar 3× no alcanza para recuperar 7.8×.

Contra su propia baseline por frontera el paralelismo sí funciona (3.03× en la i5, 2.58× en la i7), pero preferimos reportar la comparación desfavorable: el speedup honesto se mide contra el mejor secuencial disponible, no contra el más conveniente.

Cómo se arreglaría. Añadir corte anticipado a media capa también en la versión paralela, con una bandera atómica que los hilos consulten para abandonar el resto de su bloque. El costo es que el trabajo deja de ser determinista — cada corrida recorrería un número distinto de aristas según qué hilo encuentre primero el destino — y habría que relajar la verificación de invariantes para ese modo. No lo implementamos por tiempo; queda documentado y medido.

Evidencia de ejecución

4. Conclusiones

1. Reestructurar la cola FIFO en fronteras por nivel expone el paralelismo sin costo: en el recorrido completo la versión por frontera ya es 1.33× más rápida que la cola clásica en la i5 y 1.36× en la i7 antes de paralelizar, por mejor localidad de caché.
2. El CAS atómico sobre un arreglo de marcas resuelve la carrera sin ningún **critical**, y los buffers por hilo eliminan la contención al construir la frontera. Las invariantes fueron estables en las 52 corridas de ambas máquinas.
3. **dynamic** con chunk 16 es la mejor política, replicado independientemente en dos CPUs. La ganancia es de 8–11 %, no espectacular, porque con fronteras de más de un millón de nodos la varianza de grados se promedia sola.
4. Las dos optimizaciones que más rindieron no vinieron de la teoría sino de medir: una sola región paralela en vez de una por nivel, y reducir el arreglo de marcas de 8 MB a 2 MB para que quepa en L3. Esta última mejoró la baseline secuencial un 21 % — es decir, nos *quitó* speedup aparente, pero es el número honesto.
5. El resultado final es 2.50× y 3.21× sobre 4 núcleos físicos en un problema limitado por memoria. Para la consulta puntual $X \rightarrow Y$ la versión paralela no supera al BFS de cola secuencial, y dejamos la causa y la solución documentadas.

```
$ wsl.exe -d Ubuntu -- bash
"/mnt/c/Users/antho/AppData/Local/Temp/claude/C--Users-antho-Repositorios-Parcial1-Paralela/1aa1acb5-5302-4b34-a575-0fc48d238169/scratchpad/correr_aqui.sh"

=====
=====
Listo.
mediciones : docs/resultados/laptop-i5-1035g1/mediciones.csv (196 filas)
entorno     : docs/resultados/laptop-i5-1035g1/entorno.txt
salidas     : docs/resultados/laptop-i5-1035g1/salidas/ (28 archivos)

Siguiendo paso:
python3 docs/benchmarks/analizar.py
```

```
$ wsl -d Ubuntu -- bash -lc 'bash
"/mnt/c/Users/antho/AppData/Local/Temp/claude/C--Users-antho-Desktop-Parcial1-Paralela/d37a285e-05f5-4810-8f00-e1562b201c86/scratchpad/fase4.sh"
2>&1'
```

185.49 max 233.18)				
sched-dynamic-chunk0	mediana	140.17 ms	(min 131.20 prom	
144.42 max 168.19)				
sched-dynamic-chunk16	mediana	109.64 ms	(min 103.80 prom	
122.50 max 185.61)				
sched-dynamic-chunk64	mediana	112.04 ms	(min 98.08 prom	
114.56 max 129.23)				
sched-dynamic-chunk256	mediana	112.41 ms	(min 99.78 prom	
117.86 max 156.62)				
sched-guided-chunk0	mediana	119.12 ms	(min 102.91 prom	
150.49 max 264.17)				

Figura 1: Arriba: cierre del barrido en la i5-1035G1 (196 filas de mediciones, 28 corridas registradas). Abajo: barrido de políticas de scheduling en la i7-11370H, donde se observa **dynamic** con chunk 16 en 109.64 ms, la configuración ganadora.